Revision — 1.1.2 Types of Processor

OCR H446 — one-page revision summary

Must know

- **Von Neumann**: a **single shared memory and bus** for both data and instructions — simpler, cheaper, flexible, but can suffer a **bottleneck** (cannot fetch an instruction and access data at the same time). Used in general-purpose PCs.
- **Harvard**: **separate memories and buses** for data and instructions, allowing **simultaneous** instruction fetch and data access — more complex/costly. Used in embedded systems, microcontrollers and DSPs.
- Most modern CPUs are **modified (hybrid) Harvard**: one unified main memory (Von Neumann style) but **separate L1 instruction and data caches** (Harvard style) close to the core.
- **CISC**: many complex, variable-length, multi-cycle instructions; complexity is in the **hardware**; higher power; used in x86 desktops.
- **RISC**: few simple, fixed-length instructions aiming for **one cycle each**; complexity is in the **software/compiler**; easier to pipeline; lower power; used in ARM mobile/embedded devices.
- A **GPU** has **thousands of small cores** that apply the **same operation to large blocks of data in parallel** (data parallelism) — ideal for rendering pixels/vertices of 3D scenes.
- **GPGPU** = using a GPU for non-graphics work: machine-learning/neural-network training, scientific simulation, image/video processing, crypto hashing.
- **Multicore** = two or more independent cores on one chip. **Parallel processing** splits a task across cores. Benefit is **limited by the serial fraction** — data dependency, coordination overhead or an inherently sequential algorithm prevent effective parallelisation.

Key terms

Term	Definition
Von Neumann architecture	Single shared memory and bus for data and instructions
Harvard architecture	Separate memories and buses for data and instructions
CISC	Complex Instruction Set Computer — many complex instructions, complexity in hardware
RISC	Reduced Instruction Set Computer — few simple fixed-length instructions, one/cycle
GPU	Many-core processor applying one operation to large data sets in parallel
GPGPU	Using a GPU for general-purpose (non-graphics) parallel computation
Multicore processor	A chip containing two or more independent cores
Parallel processing	Carrying out multiple computations simultaneously across cores

Worked example / how to — choosing between a multicore CPU and a GPU

Scenario: apply the same calculation to millions of data points.

1. **Identify the workload type** — the same operation repeated over huge data with little branching or dependency = highly **data-parallel**.
2. **Match to the hardware** — a **GPU** has thousands of small cores, so it processes many data points **simultaneously**; a multicore CPU has fewer, more powerful cores better suited to sequential, branch-heavy work.
3. **Add the caveat** — the software must be written to use the GPU; very branchy or dependency-heavy work would suit the CPU.
4. **Conclude** — for this uniform, massively parallel task the **GPU** maximises throughput, so it is the better choice.

Exam tips

- Give **distinct** differences: memory/buses, bottleneck vs simultaneous access, cost/complexity, typical uses — don't repeat the same point reworded.
- Link processor type to a **real use**: RISC/ARM → mobile & embedded (low power); CISC/x86 → desktops; GPU → graphics/GPGPU.
- Explain GPU speed with **two** ideas: **thousands of cores** *and* **the same operation on many data items in parallel**.
- For parallelism questions, name the reason benefit is limited — **data dependency / serial fraction** — and note software must support multiple threads.